DUOC UC - ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Documento de Cobertura de Testing

Proyecto: Pan del Sur

Revisión: [02.00]

25/10/2026

Documento de cobertura de pruebas automatizadas del frontend (Jasmine y Karma).

-

Contenido

Ficha del documento 4

1. Objetivo 5

2. Alcance 5

3. Herramientas y entorno de pruebas 6

3.1. Configuración del entorno 6
3.2. Comandos utilizados 6
3.3. Entorno de ejecución verificado 7

4. Estrategia de testeo y conceptos aplicados 7

4.1. Pruebas de componentes 7
4.2. Patrón Arrange, Act, Assert 7
4.3. Aislamiento de las pruebas 7
4.4. Funciones auxiliares 8
4.5. Uso de act 8
4.6. Tipos de escenarios 8

5. Mocks y dobles de prueba 8

6. Inventario de pruebas implementadas 8

6.1. Pruebas de componentes.spec.jsx 9
6.2. Pruebas de pantallas.spec.jsx 9

7. Matriz de cobertura funcional 10

8. Cobertura de código 10

8.1. Procedimiento de ejecución 10
8.2. Resultados de la ejecución original de 22 pruebas 11
8.3. Cobertura por carpeta 11
8.4. Cobertura por archivo de servicios 12

9. Análisis de resultados 12

9.1. Interpretación general 12
9.2. Análisis por carpeta 12
9.3. Archivos con menor cobertura 13
9.4. Limitaciones identificadas 13
9.5. Mejoras futuras 13

10. Correspondencia con la rúbrica 14

11. Conclusión 14

Anexo: Evidencia de ejecución 15

-

Ficha del documento

Fecha	Revisión	Autor	Modificación
25/10/2026	02.00	Equipo de desarrollo Pan del Sur	Elaboración del documento de cobertura de testing de la Evaluación Parcial 2: herramientas, estrategia, mocks, 22 casos de prueba y análisis de cobertura.

Datos generales

Dato	Detalle
Proyecto	Pan del Sur — Sistema de Producción y Control de Calidad
Asignatura	DSY1104 — Desarrollo Fullstack II
Evaluación	Evaluación Parcial N.º 2
Versión	02.00
Fecha	25 de octubre de 2026
Rama de GitHub	`evaluacion-2-react`

-

1. Objetivo

El presente documento tiene como objetivo describir, verificar y analizar el proceso de pruebas automatizadas implementado en el frontend de Pan del Sur, desarrollado mediante React.

Se documentan las herramientas utilizadas, la configuración del entorno, la estrategia de testing, los mocks, los 22 casos de prueba implementados y los resultados obtenidos mediante el análisis de cobertura de código.

Asimismo, se identifican las funcionalidades verificadas, los aspectos que no forman parte del alcance de las pruebas y las oportunidades de mejora detectadas.

El documento permite demostrar que se ha implementado un proceso de testing reproducible, orientado a verificar el comportamiento de los componentes y las principales reglas de negocio del sistema.

2. Alcance

El proceso de testing considera la aplicación frontend desarrollada con React 19, React Router 7, Bootstrap 5 y Vite.

Elementos incluidos:

- Componentes de estructura: `Header` y `Navbar`.
- Pantallas: `Inicio`, `ProductosIngredientes`, `OrdenesProduccion`, `ControlCalidad`, `Trazabilidad` y `Reportes`.
- Funciones de gestión y validación ubicadas en `src/services/`, ejecutadas principalmente mediante las pantallas.
- Interacciones con el DOM, formularios, eventos, mensajes de error, tablas y búsquedas.
- Operaciones de persistencia mediante `localStorage`, utilizando mocks para mantener aisladas las pruebas.
- Reglas de negocio asociadas a órdenes de producción, control de calidad, productos, inventario y reportes.

Elementos fuera del alcance:

- Pruebas end-to-end completas.
- Pruebas automatizadas de responsividad y apariencia visual.
- Pruebas de carga y rendimiento.
- Pruebas directas de `Footer.jsx`, `App.jsx` y `main.jsx`.
- Pruebas aisladas de todas las funciones disponibles en `src/services/`.

-

3. Herramientas y entorno de pruebas

Elemento	Herramienta	Función
Framework de pruebas	Jasmine	Definición de casos, expectativas y espías
Ejecutor	Karma	Ejecución automatizada de las pruebas
Navegador	ChromeHeadless	Ejecución en Chrome sin interfaz gráfica
Empaquetado	Webpack y karma-webpack	Compilación del código y archivos de prueba
Transpilación	Babel	Compatibilidad con JSX y JavaScript moderno
Cobertura	Istanbul y karma-coverage	Instrumentación y generación de reportes
Renderizado	React DOM y `act`	Montaje y actualización de componentes
Navegación	MemoryRouter	Simulación del contexto de navegación

3.1. Configuración del entorno

El proyecto dispone de un archivo `karma.conf.cjs` que configura el entorno de ejecución.

Los archivos de pruebas se encuentran en la carpeta `tests/` y utilizan el patrón `tests/**/*.spec.jsx`.

La configuración permite compilar los archivos mediante Webpack, utilizar Babel para procesar JSX y ejecutar las pruebas en ChromeHeadless.

También contempla el reemplazo de `import.meta.env.BASE_URL` para su compatibilidad con Webpack y la generación de reportes de cobertura mediante Istanbul.

La ejecución utiliza el modo `singleRun`, lo que permite iniciar las pruebas, mostrar sus resultados y finalizar el proceso automáticamente.

3.2. Comandos utilizados

Comando	Descripción
`npm.cmd install`	Instala las dependencias
`npm.cmd run dev`	Ejecuta el frontend en desarrollo
`npm.cmd test`	Ejecuta las pruebas y genera la cobertura
`npm.cmd run test:watch`	Ejecuta las pruebas en modo continuo
`npm.cmd run build`	Compila la aplicación para producción

Se utiliza `npm.cmd` porque PowerShell puede restringir la ejecución del archivo `npm.ps1`. En terminales sin esa restricción, puede utilizarse `npm`.

-

3.3. Entorno de ejecución verificado

Elemento	Versión o resultado
Sistema operativo	Windows
Node.js	24.21.0
npm	11.19.0
Karma	6.4.4
Webpack	5.112.0
ChromeHeadless	155.0.0.0
Vite	8.3.4
Compilación de producción	Exitosa

4. Estrategia de testeo y conceptos aplicados

4.1. Pruebas de componentes

Las pruebas verifican el comportamiento observable de los componentes React, mediante su renderizado y la simulación de interacciones.

Se comprueba que los componentes reciban y representen correctamente sus propiedades, respondan a eventos y actualicen el DOM de acuerdo con los cambios de estado.

Algunos casos ejercitan varias funciones y servicios mediante una pantalla, por lo que también tienen características de pruebas de integración a nivel de componente.

4.2. Patrón Arrange, Act, Assert

Se utiliza el patrón AAA:

- **Arrange:** preparar los datos, mocks y componentes.
- **Act:** ejecutar una interacción, como escribir, seleccionar o guardar.
- **Assert:** comprobar el resultado esperado mediante las expectativas de Jasmine.

Este enfoque facilita la comprensión y mantenimiento de los casos.

4.3. Aislamiento de las pruebas

Se utilizan `beforeEach` y `afterEach` para preparar y limpiar el entorno.

Los componentes se montan en contenedores independientes y se desmontan después de cada caso.

Las operaciones de almacenamiento se simulan mediante mocks, evitando que los resultados dependan de datos persistidos por pruebas anteriores.

-

4.4. Funciones auxiliares

Los archivos de prueba incorporan funciones auxiliares para renderizar componentes, modificar campos, enviar formularios, pulsar botones y generar fechas relativas.

Estas funciones reducen la duplicación de código y permiten mantener una estructura uniforme.

4.5. Uso de act

Las interacciones con los componentes se realizan mediante `act`, con el propósito de procesar las actualizaciones de React antes de comprobar el estado del DOM.

4.6. Tipos de escenarios

Las pruebas consideran escenarios exitosos, validaciones, errores y reglas de negocio.

Entre ellos se incluyen el registro correcto de órdenes, la prevención de códigos duplicados, la validación de rangos de humedad, la búsqueda de registros y el manejo de errores de almacenamiento.

5. Mocks y dobles de prueba

Mock o herramienta	Uso	Propósito
`spyOn(Storage.prototype, 'getItem')`	Pruebas de almacenamiento	Simular lectura de datos
`spyOn(Storage.prototype, 'setItem')`	Pruebas de almacenamiento	Simular escritura y verificar llamadas
`spyOn(Storage.prototype, 'removeItem')`	Pruebas de componentes	Simular eliminación de datos
`setItem` con error simulado	Prueba 10	Verificar el manejo de fallos
`spyOn(window, 'confirm')`	Prueba 09	Simular aceptación y cancelación
`MemoryRouter`	Pruebas 02 y 13	Proporcionar contexto de navegación
`DefinePlugin`	Configuración de Webpack	Adaptar una variable propia de Vite

El uso de mocks permite controlar las condiciones de ejecución y verificar situaciones que serían difíciles de reproducir de manera consistente utilizando el navegador y el almacenamiento real.

6. Inventario de pruebas implementadas

Se dispone de **22 casos de prueba**, distribuidos en dos archivos:

Archivo	Cantidad
`tests/componentes.spec.jsx`	12
`tests/pantallas.spec.jsx`	10
Total	**22**

-

6.1. Pruebas de componentes.spec.jsx

N.º	Componente	Comportamiento verificado	Tipo
01	Header	Muestra el nombre recibido y el texto alternativo del logo	Props y DOM
02	Navbar	Muestra seis enlaces y marca la ruta `/calidad` como activa	Navegación
03	OrdenesProduccion	Actualiza el campo controlado de código	Estado
04	OrdenesProduccion	Reinicia el producto y filtra opciones al cambiar categoría	Estado y lógica
05	OrdenesProduccion	Rechaza una orden vacía y muestra errores	Validación
06	OrdenesProduccion	Guarda una orden válida y limpia el formulario	Persistencia
07	OrdenesProduccion	Actualiza una orden sin duplicarla	Edición
08	OrdenesProduccion	Cancela una edición conservando los datos	Estado
09	OrdenesProduccion	Solicita confirmación antes de eliminar	Mock y persistencia
10	OrdenesProduccion	Maneja un fallo de almacenamiento	Manejo de errores
11	ControlCalidad	Rechaza el resultado Conforme con humedad fuera de rango	Regla de negocio
12	ControlCalidad	Exige observaciones para No conforme	Validación

6.2. Pruebas de pantallas.spec.jsx

N.º	Pantalla	Comportamiento verificado	Tipo
13	Inicio	Muestra tres cotizadores y calcula un total de $3.000 para dos unidades	Cálculo
14	ProductosIngredientes	Muestra seis productos y permite filtrarlos	Búsqueda
15	ProductosIngredientes	Rechaza un código de producto duplicado	Validación
16	ProductosIngredientes	Guarda un producto nuevo y actualiza el listado	Persistencia
17	ProductosIngredientes	Muestra ocho ingredientes y detecta bajo stock de mantequilla	Regla de negocio
18	Trazabilidad	Encuentra información del lote mediante un código de insumo	Búsqueda
19	Trazabilidad	Consulta inspecciones almacenadas sin distinguir mayúsculas	Búsqueda
20	Reportes	Rechaza un período con fechas invertidas	Validación
21	Reportes	Filtra órdenes dentro de un período	Filtrado
22	Reportes	Muestra el reporte de ingredientes sin aplicar filtro de fechas	Condicional de interfaz

-

7. Matriz de cobertura funcional

Módulo o archivo	Pruebas asociadas	Funcionalidad verificada	Alcance
`Header.jsx`	01	Propiedades y renderizado	Cubierto en el escenario evaluado
`Navbar.jsx`	02	Enlaces y ruta activa	Cubierto en el escenario evaluado
`Footer.jsx`	—	Sin prueba directa	No probado directamente
`Inicio.jsx`	13	Cotizadores y cálculo	Parcial
`ProductosIngredientes.jsx`	14–17	Catálogo, búsqueda, registro y stock	Parcial
`OrdenesProduccion.jsx`	03–10	Formulario, validación, CRUD y errores	Amplio
`ControlCalidad.jsx`	11–12	Validaciones y registro	Parcial
`Trazabilidad.jsx`	18–19	Búsqueda de lotes e inspecciones	Parcial
`Reportes.jsx`	20–22	Validación de fechas y filtrado	Parcial
`services/validacionOrdenes.js`	05–06	Reglas utilizadas por el formulario	Indirecto
`services/validacionCalidad.js`	11–12	Reglas utilizadas por el formulario	Indirecto
`services/catalogo.js`	14–17	Gestión de productos e ingredientes	Indirecto
`services/lotes.js`	18	Información de lotes	Indirecto
`services/ordenes.js`	06–10, 21	Gestión y consulta de órdenes	Indirecto
`services/inspecciones.js`	12, 19	Registro y consulta de inspecciones	Indirecto
`App.jsx`	—	Definición general de rutas	No probado directamente
`main.jsx`	—	Arranque de React	Fuera del alcance

La clasificación anterior corresponde a la cobertura funcional de los casos documentados y no representa un porcentaje de cobertura de código por archivo.

8. Cobertura de código

La cobertura se obtiene mediante Istanbul durante la ejecución de Jasmine y Karma.

El sistema genera un resumen en consola y un informe HTML que permite identificar sentencias, ramas, funciones y líneas ejecutadas.

-

8.1. Procedimiento de ejecución

Desde la carpeta `frontend`:

1. Instalar dependencias mediante `npm.cmd install`.
2. Ejecutar las pruebas mediante `npm.cmd test`.
3. Revisar el resultado mostrado en la terminal.
4. Abrir `coverage/html/index.html` para consultar el detalle.
5. Analizar los archivos con menor cobertura.

8.2. Resultados de la ejecución original de 22 pruebas

Fecha de ejecución: 25 de octubre de 2026

Herramientas: Jasmine, Karma, ChromeHeadless e Istanbul.

Indicador	Resultado
Pruebas ejecutadas	22
Pruebas exitosas	22
Pruebas fallidas	0
Porcentaje de aprobación	100 %
Statements	74,46 % (452/607)
Branches	69,44 % (391/563)
Functions	76,59 % (144/188)
Lines	74,78 % (436/583)

Estos valores corresponden a la ejecución verificada de los 22 casos. Posteriormente se reorganizó la carpeta `data` como `services`, sin modificar la lógica de sus ocho archivos. Después de reorganizar la carpeta `data` como `services` y conservar únicamente los 22 casos originales, se ejecutó nuevamente el conjunto de pruebas, confirmando su funcionamiento satisfactorio (22 pruebas exitosas) y la continuidad del proceso de testing.

-

8.3. Cobertura por carpeta

Carpeta actual	Statements	Branches	Functions	Lines
components	100 %	N/A (0/0)*	100 %	100 %
services (antes data)	70,55 %	61,90 %	66,66 %	70,89 %
pages	76,22 %	72,65 %	79,43 %	76,53 %
Total	**74,46 %**	**69,44 %**	**76,59 %**	**74,78 %**

** En components no se contabilizaron ramas condicionales (0/0); por lo tanto, no corresponde interpretar un porcentaje de ramas y no significa que se hayan probado todas las condiciones.*

Los valores de services proceden del informe generado cuando la carpeta todavía se denominaba data. Se utilizan como referencia porque el cambio posterior fue de organización y rutas, no de lógica.

8.4. Cobertura por archivo de servicios

Archivo	Statements	Branches	Functions	Lines
catalogo.js	56,09 %	44 %	50 %	57,89 %
inspecciones.js	42,85 %	30 %	33,33 %	44,44 %
lotes.js	100 %	N/A (0/0)*	N/A (0/0)*	100 %
nombresProductos.js	100 %	50 %	100 %	100 %
ordenes.js	89,28 %	60 %	100 %	88 %
productosDisponibles.js	100 %	75 %	100 %	100 %
validacionCalidad.js	70,58 %	59,72 %	75 %	70,58 %
validacionOrdenes.js	84,21 %	83,72 %	40 %	84,21 %

** En lotes.js no se contabilizaron ramas ni funciones evaluables (0/0).*

Los resultados presentados corresponden al conjunto de 22 pruebas automatizadas que forman parte de la versión definitiva del proyecto para la Evaluación Parcial N.º 2.

-

9. Análisis de resultados

9.1. Interpretación general

La ejecución original de las 22 pruebas finalizó correctamente, sin fallos.

Esto demuestra que los casos implementados verificaron satisfactoriamente los comportamientos definidos en sus respectivas expectativas.

La cobertura de sentencias alcanzó un 74,46 %, mientras que la cobertura de líneas obtuvo un 74,78 %. Ambos resultados indican que aproximadamente tres cuartas partes del código instrumentado fueron ejecutadas durante las pruebas.

La cobertura de funciones alcanzó un 76,59 %, lo que demuestra que una proporción importante de las funciones instrumentadas fue utilizada.

La cobertura de ramas obtuvo un 69,44 %, siendo el indicador general más bajo. Esto evidencia que todavía existen condiciones alternativas, escenarios de error y caminos de ejecución no ejercitados.

9.2. Análisis por carpeta

La carpeta `components` presenta cobertura completa del código instrumentado, aunque el informe no registra ramas condicionales evaluables.

La carpeta `pages` presenta una cobertura de sentencias del 76,22 % y una cobertura de funciones del 79,43 %, reflejando la ejecución de una parte importante de la lógica de las pantallas.

La carpeta `services` presenta los valores más bajos, especialmente en ramas y funciones. Esto se relaciona con la existencia de operaciones de gestión y validación que no cuentan con pruebas directas para todos sus escenarios.

9.3. Archivos con menor cobertura

Los archivos con menor cobertura de sentencias son:

- `inspecciones.js`: 42,85 %.
- `catalogo.js`: 56,09 %.
- `validacionCalidad.js`: 70,58 %.

En el caso de `validacionOrdenes.js`, aunque la cobertura de sentencias alcanza un 84,21 %, solamente el 40 % de las funciones fue ejecutado.

Estos resultados permiten identificar oportunidades de mejora sin afectar la validez de las pruebas que actualmente se encuentran implementadas.

-

9.4. Limitaciones identificadas

El proceso de testing presenta las siguientes limitaciones:

- No existen pruebas directas para `Footer.jsx` ni para las rutas generales de `App.jsx`.
- No se verifica la edición y eliminación de productos e ingredientes en todos sus escenarios.
- No se verifica la edición y eliminación de inspecciones de calidad.
- No se cubren todos los escenarios de cantidades inválidas en el cotizador.
- No se comprueba la exportación de reportes a CSV.
- Las funciones de validación se ejercitan principalmente mediante las interfaces.
- No se realizan pruebas automatizadas de diseño responsivo.
- No se incluyen pruebas end-to-end completas.

9.5. Mejoras futuras

Como posibles mejoras posteriores se identifican la incorporación de pruebas aisladas de validación, la comprobación de escenarios límite, la ampliación de casos de edición y eliminación y la verificación de las rutas generales de navegación.

También podría establecerse un umbral mínimo de cobertura para detectar disminuciones en futuras versiones.

Estas mejoras se consideran propuestas para una evolución posterior del proyecto y **no forman parte de las 22 pruebas definidas para esta evaluación**.

10. Correspondencia con la rúbrica

Indicador	Evidencia
IE2.2.1 — Crea pruebas unitarias para componentes frontend utilizando Jasmine y Karma	22 casos implementados en dos archivos, que superan el mínimo de 10 pruebas
IE2.3.1 — Implementa un proceso de testing unitario	Configuración de Karma, uso de Jasmine, mocks, aislamiento, ejecución y análisis de resultados
Verificación de lógica y comportamiento	Validaciones, formularios, persistencia, búsquedas, filtrados y cálculos
Manipulación del DOM	Renderizado, actualización de campos, tablas y mensajes
Análisis de cobertura	Informe de Istanbul con porcentajes generales y por archivo
Evidencia de funcionamiento	Ejecución original con 22 pruebas exitosas y compilación de producción exitosa

-

11. Conclusión

El frontend de Pan del Sur dispone de un entorno de pruebas automatizadas implementado mediante Jasmine y Karma, integrado con ChromeHeadless, Webpack, Babel e Istanbul.

Se desarrollaron 22 casos de prueba distribuidos entre componentes y pantallas, orientados a verificar la lógica de negocio, las validaciones, el comportamiento de la interfaz y las operaciones de persistencia local.

La ejecución original de los 22 casos finalizó satisfactoriamente, alcanzando un 100 % de aprobación y una cobertura de sentencias del 74,46 %, de ramas del 69,44 %, de funciones del 76,59 % y de líneas del 74,78 %.

Adicionalmente, se reorganizó la estructura del proyecto, reemplazando la carpeta `data` por `services`, con el propósito de representar mejor las responsabilidades de gestión y validación de sus archivos. La compilación de producción posterior a esta reorganización fue exitosa.

En conclusión, el proyecto cuenta con un proceso de testing funcional, documentado y reproducible, que proporciona evidencia sobre los comportamientos evaluados y permite identificar oportunidades de mejora para futuras versiones.

Los resultados obtenidos evidencian el cumplimiento del alcance de pruebas establecido para esta evaluación, manteniendo una base de 22 casos automatizados que permite verificar funcionalidades relevantes y facilitar la detección de errores durante el desarrollo.

-

Anexo: Evidencia de ejecución

Este anexo reúne la evidencia visual de la ejecución de las pruebas, para contrastar los resultados documentados con la ejecución real.

Figura A.1. Resultado de `npm.cmd test` en la terminal.

```
PS C:\Users\aleja\Downloads\PanDelSur-React-Actualizado\frontend> npm.cmd test

> frontend@0.0.0 test
> karma start karma.conf.cjs --single-run

Webpack bundling...
asset commons.js 4.94 MiB [emitted] (name: commons) (id hint: commons)
asset runtime.js 17.2 KiB [emitted] (name: runtime)
asset componentes.spec.278402593.js 440 bytes [emitted] (name: componentes.spec.278402593)
asset pantallas.spec.244005843.js 436 bytes [emitted] (name: pantallas.spec.244005843)
Entrypoint componentes.spec.278402593 4.95 MiB = runtime.js 17.2 KiB commons.js 4.94 MiB componentes.spec.278402593.js 440 bytes
Entrypoint pantallas.spec.244005843 4.95 MiB = runtime.js 17.2 KiB commons.js 4.94 MiB pantallas.spec.244005843.js 436 bytes
webpack 5.112.0 compiled successfully in 10813 ms
25 10 2026 01:16:47.985:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
25 10 2026 01:16:47.986:INFO [launcher]: Launching browsers ChromeHeadless with concurrency unlimited
25 10 2026 01:16:48.035:INFO [launcher]: Starting browser ChromeHeadless
25 10 2026 01:16:48.925:INFO [Chrome Headless 155.0.0.0 (Windows 10)]: Connected on socket a8P_8tWr4m0-LBTMAAAA with id 95699968
Chrome Headless 155.0.0.0 (Windows 10): Executed 0 of 22 SUCCESS (0 secs / 0 secs)
(node:7248) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.
Chrome Headless 155.0.0.0 (Windows 10): Executed 22 of 22 SUCCESS (0.41 secs / 0.392 secs)
TOTAL: 22 SUCCESS

============================== Coverage summary ==============================
Statements   : 74.46% ( 452/607 )
Branches     : 69.44% ( 391/563 )
Functions    : 76.59% ( 144/188 )
Lines        : 74.78% ( 436/583 )
==============================================================================
PS C:\Users\aleja\Downloads\PanDelSur-React-Actualizado\frontend>
```

Figura A.2. Informe HTML de Istanbul (`coverage/html/index.html`).

